Cordell Programming Language and Compiler: Restricted Syntax, Explicit Low-Level Control, and Static Diagnostics for Systems Programming

Fot Nikolai

April 2026

Abstract

This technical report presents the Cordell Programming Language (CPL) and the Cordell Compiler, an experimental systems-programming language and end-to-end compiler pipeline. The central design choice is a deliberately restricted language surface: the language keeps C-like low-level control, pointer operations, inline assembly, explicit casts, and compiler annotations, while avoiding a large object model and complex aggregate abstractions. This restriction is used to keep parsing, lowering, static diagnostics, and backend experimentation tractable. The report describes the language core, preprocessing and lexical stages, semantic markup, AST construction, HIR lowering, CFG and SSA construction, DAG-based optimizations, liveness analysis, register allocation, peephole optimization, and NASM-oriented code generation. It also summarizes the diagnostic model and preliminary benchmark measurements against GCC 14.2.0 and Apple Clang 12.0.0. The reported results should be interpreted as project-level evidence for selected microbenchmarks rather than as an independent compiler evaluation.

Keywords: systems programming language; compiler construction; static analysis; SSA; register allocation; peephole optimization; low-level programming.

1 Introduction

Systems languages often expose low-level control at the cost of a large and historically accumulated surface syntax. This makes them practical for production work, but it also makes them difficult to use as compact experimental platforms for compiler construction. The Cordell Programming Language (CPL) addresses a different point in the design space. It is a small systems language whose primary purpose is to support low-level programming experiments, compiler-pipeline research, and educational work on whole-compiler architecture.

The project targets programs such as bootloaders, small operating-system components, edge database-management systems, edge file systems, simple compilers, and interpreters. It does not attempt to provide memory safety in the style of modern ownership-based languages. Instead, the design emphasizes explicit behavior: pointer operations are visible, casts are written by the programmer when narrowing may occur, and low-level mechanisms such as system calls and inline assembly are exposed directly.

The second motivation is compiler-oriented. A smaller grammar makes it easier to construct a complete pipeline, inspect intermediate forms, and experiment with classic middle-end and backend techniques such as control-flow construction, dominance analysis, SSA conversion, common subexpression elimination, liveness analysis, graph-coloring register allocation, and peephole rewriting [1, 2, 3, 4].

This report makes the following contributions:

- it gives a consolidated article-style description of the CPL language core and its low-level mechanisms;
- it presents the compiler pipeline from preprocessing to NASM-oriented code generation;
- it describes how a restricted grammar is used together with annotations rather than a large set of language constructs;
- it summarizes static diagnostics at both AST and intermediate-representation levels;
- it reports preliminary benchmark values and states the limitations of these measurements.

2 Design Rationale

2.1 Restricted language surface

The central language-design decision is to keep the source language small. CPL deliberately omits structures, classes, enums, unions, and a rich standard-library-oriented object model. This is not meant to make the language more expressive than existing systems languages. Instead, it narrows the front-end problem and shifts the project focus toward intermediate representations, optimizations, diagnostics, and backend generation.

This restriction has three practical effects. First, the parser and semantic passes stay compact enough to be inspected and modified. Second, the compiler can expose transformations explicitly, which is useful for learning and debugging. Third, the absence of high-level aggregates reduces the number of special lowering cases in the current implementation.

2.2 Explicit control instead of implicit safety

The language is low-level and intentionally unsafe. It gives direct access to pointer operations, dereferencing, explicit casts, system calls, and inline assembly. This choice makes CPL closer to an experimental C-like compiler laboratory than to a memory-safe language. Diagnostics are therefore used not as a formal safety guarantee, but as a practical mechanism for detecting suspicious operations and common low-level mistakes.

2.3 Annotations as controlled extensions

Rather than adding many special syntactic constructs, CPL uses annotations for selected behavior: entry-point selection, branch-layout hints, non-fallthrough switches, counted loops, alignment, section placement, and register binding. This keeps the core grammar relatively small while still allowing low-level code-generation control.

3 Language Core

3.1 Primitive types and casts

CPL is statically typed but permissive in the direction of safe widening conversions. Narrowing conversions are never implicit and must be written with `as`. The same cast syntax can be used when overload resolution requires a specific argument type.

Primitive types include signed and unsigned integers from 8 to 64 bits, `f32`, `f64`, and `i0`. The `i0` type acts as a void-like type for function returns and as a generic pointed-to type in `ptr i0`. There is no separate Boolean type: zero represents false and non-zero values represent true.

Listing 1: Explicit casts and pointer operations.

```
i32 a = 10 as i32;
u8  b = (a + 20 as i32) as u8;
ptr i32 p = ref a;
dref p = 11 as i32;
```

The main non-primitive containers are `str` and `arr`. A stack string behaves similarly to an array of `i8` with a terminating zero byte. Independent string literals are stored in a read-only section and must be referred to with `ref`. Arrays contain primitive element types and are used with explicit indexing and pointer-style operations.

3.2 Control flow

The control-flow syntax is C-like but intentionally explicit. An `if` condition is followed by a semicolon. `while` is the ordinary conditional loop. `loop` is an endless loop; it must either contain `break` or be

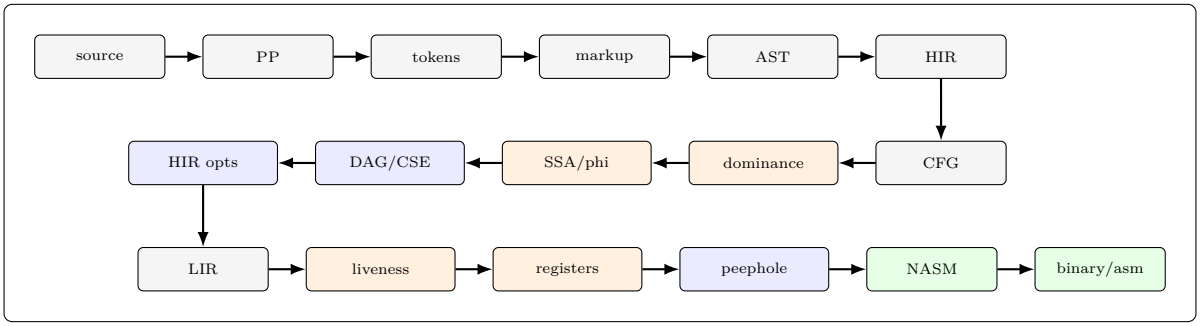

Figure 1: Compiler pipeline implemented around the restricted CPL language core.

controlled by `@[counter(...)]`. The `switch` construct supports C-style fallthrough. The `@[no_fall]` annotation inserts hidden breaks into cases, while `@[straight]` selects a linear search strategy for switch lowering.

Listing 2: Counted loop and non-fallthrough switch.

```
@[counter(10)] loop {
    : repeated ten times :
}

@[no_fall]
@[straight]
switch tag; {
    case 1; { }
    default { }
}
```

Listing 3: Hidden break in a prepared switch.

```
  i32s %0 = alloc(8);
  goto lb9;
  lb11:
  {
  }
  goto lb10; // break
  lb16:
  {
  }
  goto lb10; // break
  lb9:
  i64t %2 = csu8 1 as i64;
  i64t %3 = i32s %0 as i64;
  i8t %1 = i64t %3 == i64t %2;
  if i8t %1, goto lb11, else goto lb23;
  lb23:
  goto lb16;
  lb10:
```

3.3 Functions and local functions

Functions are declared with `function`. The language documentation includes prototypes, default arguments, overloading, local functions, function pointers, lambda functions, and variadic arguments. Overloading is restricted: return-type-only overloading is forbidden, overloads are local-only, and candidates are searched in the same scope. Function pointers use `ptr i0`; no signature information remains available through such a pointer.

Local functions and lambdas do not capture the parent environment. They are therefore lightweight logic containers rather than closures. This design keeps lowering simple and avoids introducing hidden environment objects.

Listing 4: Local function passed as a pointer.

```
function apply(ptr i0 f, i32 a, i32 b) {
    f(a, b);
}

@[entry("_start")] function main() {
    function add(i32 a, i32 b) {
        return a + b;
    }
    apply(add, 10 as i32, 20 as i32);
    exit 0;
}
```

Table 1: Documented annotation roles.

Annotation	Role
`naked`	disable entry/exit routines
`align`	align variables or arrays
`section`	place functions/globals in sections
`address`	place a function at an address
`entry`	mark entry point
`no_fall`	add hidden breaks in switch
`not_lazy`	generate both logic operands
`straight`	use linear switch search
`counter`	make counted loop
`hot`, `cold`	control branch placement
`register`	bind local variable to register

4 Annotations and Low-Level Mechanisms

Table 1 summarizes the documented annotations. These annotations provide low-level control without substantially extending the base grammar.

Two built-in mechanisms expose behavior below the ordinary language level: `syscall` and `asm`. A syscall is invoked like a function and takes a platform-dependent argument list. Inline assembly can receive CPL values and refer to them through numbered placeholders. Inline assembly is not optimized by the compiler and should be treated as an opaque operation during optimization.

5 Front End

5.1 Preprocessing

The preprocessor supports `include`, `define`, `undef`, `ifdef`, and `ifndef`. This is necessary because the project uses C-like header organization: headers contain prototypes and guards, while implementation files contain function bodies. After preprocessing, line directives preserve source-origin information for the tokenizer and later diagnostics.

5.2 Tokenization and semantic markup

Tokenization splits the input byte sequence into tokens using deterministic scanning logic. Semantic markup is a second token-level stage. It uses scopes and declarations to attach semantic information to tokens before full parsing. For example, if `i32 a` is declared in a scope, later visible occurrences of `a` can be marked as references to an `i32` entity. This stage is not a replacement for type checking, but it gives the parser and later diagnostics a richer token stream.

5.3 AST construction

The parser builds an AST using an LL-style approach. Keeping the grammar restricted reduces ambiguity and keeps recursive-descent parsing practical. AST-level transformations include condition unrolling and dead-scope elimination, but the main optimization path starts after lowering to an intermediate representation.

6 Intermediate Representations

6.1 HIR lowering

The AST is flattened into a three-address-code-like high-level intermediate representation. This representation makes evaluation order and temporary values explicit. For example, array indexing is expanded into base-address computation, scaled offsets, address addition, and memory store operations, as shown in Figure 2.

6.2 CFG and SSA

The CFG is generated from HIR. The implementation supports both one-block-per-command construction and leader-based splitting. In the leader approach, a new block starts at the first instruction, at jump targets, and immediately after jumps.

The compiler then computes dominators, strict dominators, and dominance frontiers to construct SSA form. Each assignment creates a new variable version, and phi functions are inserted where values

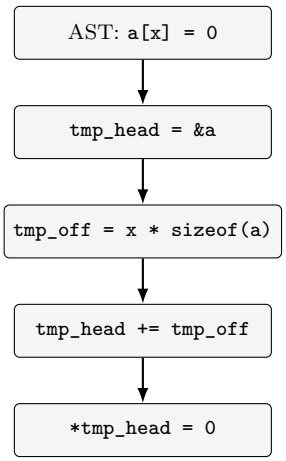

Figure 2: AST-to-HIR flattening into explicit address operations.

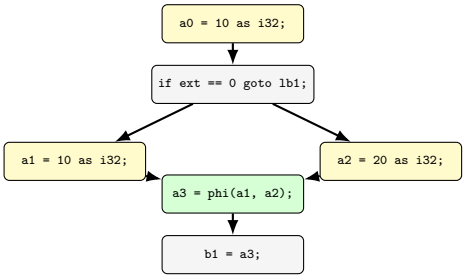

Figure 3: SSA merge point with a phi function.

from distinct control-flow paths merge [4]. Figure 3 illustrates this form.

7 Optimization Pipeline

After SSA construction, the compiler can build a DAG that records how values are derived. Nodes with the same hash and compatible dominance relation can be merged, enabling common subexpression elimination. Other HIR optimizations include constant folding, constant propagation, dead function elimination through a call graph, tail-recursion elimination, function inlining, loop canonicalization, and loop-invariant code motion.

Inlining uses a heuristic score. Calls in loops and

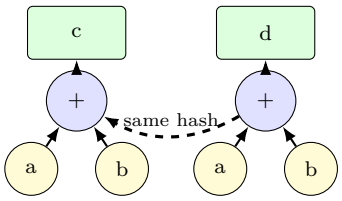

Figure 4: DAG-based common subexpression elimination.

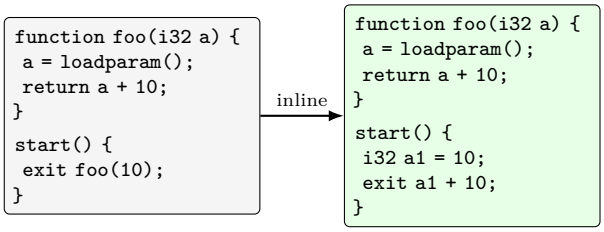

Figure 5: Function inlining by body copying and return substitution.

calls to small functions receive a positive score; large callees receive a negative score. If the score passes the threshold, the compiler copies the callee body, replaces argument loads, and substitutes the returned value.

8 LIR, Liveness, and Register Allocation

LIR is a lower intermediate representation closer to target instructions. It is similar to three-address code, but it uses a two-address form where appropriate. It bridges the higher-level HIR and target-specific assembly generation. On x86-64, this is the stage where instruction selection, ABI-related pre-coloring, and register allocation become important.

Liveness analysis computes `USE`, `DEF`, `IN`, and `OUT` for each CFG block:

$$OUT[B] = \bigcup_{S \in succ(B)} IN[S]$$

$$IN[B] = USE[B] \cup (OUT[B] - DEF[B]).$$

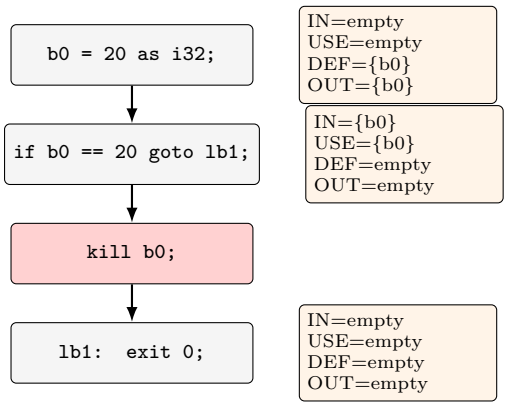

Figure 6: Liveness-driven kill insertion.

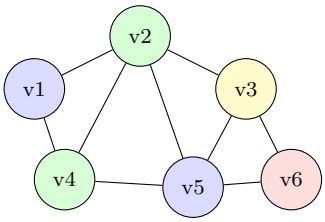

Figure 7: Interference graph coloring for register allocation.

The compiler iterates in reverse order until the sets stabilize. If a variable appears in `USE` or `DEF` but not in `OUT`, the compiler can mark it dead with a special `kill` instruction.

For pointers, the compiler uses an alias map. If `b0` owns a reference to `a0`, `a0` must not be killed while `b0` remains live. This is stack-slot reuse and liveness protection, not a formal memory-safety system.

Register allocation builds an interference graph. Vertices are variables; edges connect variables that are live at the same time. Coloring assigns different colors to adjacent nodes, and each color corresponds to a reusable physical register choice [5].

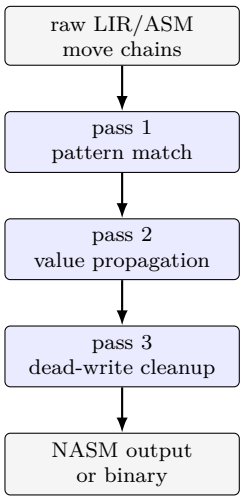

Figure 8: Documented peephole optimization structure.

9 Peephole Optimization and Code Generation

The peephole optimizer performs local pattern matching and replacement on LIR or assembly-like forms. A typical example is replacing `mov rax, 0` with `xor rax, rax`. The project uses a PTRN domain-specific language to generate pattern matchers instead of coding every pattern by hand.

The documented peephole stage has three passes: pattern matching, value propagation through redundant move chains, and dead-write cleanup. Code generation then unwraps special LIR operations such as `EXITOP` and `STRT` and emits NASM assembly.

10 Static Diagnostics and Debugging

The static checker has both AST and intermediate-representation components. AST-level warnings include read-only updates, declarations without initialization, invalid declaration values, missing returns, missing `exit` in `start`, wrong argument counts, wrong argument types, unused function results, return-type

Table 2: Documented benchmark environment.

Item	Value
OS	MacOS Catalina 10.15.7
CPU	i7 3650QM, quad-core, 2.4 GHz
RAM	16 GB DDR3, 1600 MHz
GCC	GCC 14.2.0, Homebrew
Clang	Apple Clang 12.0.0
CPL	CPL v3.4, MACHO64
Runs	5 per configuration
CPL timing	`py-time`, after compilation
C/C++ timing	`gnu-time`, after compilation

mismatches, illegal array access, duplicated branches, reserved function names, dead code, suspicious conversions, inefficient `while 1`, invalid `exit`, targetless `break`, misuse of `i0`, unused expressions, invalid `ref`, and non-even alignment.

Intermediate-representation analysis handles lower-level warnings such as null dereference, null dereference through arguments, constant conditions, propagated constants, wrong casts, and syscall-related checks. Static analysis is off by default and is enabled with `-static-analysis`.

For debugging, the compiler can produce assembly with `-emit-asm`. The `-d` flag includes DWARF information, and `lis` is an interrupt point usable with `gdb` or `lldb`. The documentation recommends disabling optimizations when debugging code that must not be transformed.

11 Benchmark Methodology

The benchmark documentation compares GCC 14.2.0, Apple Clang 12.0.0, and CPL v3.4 on MACHO64. GCC and Clang use `-O0` and `-O3`. CPL also uses `-O0` and `-O3`; the documented `-O3` set includes loop-invariant code motion, peephole optimization, common subexpression elimination, dead-code elimination, constant propagation, and constant folding.

Some C microbenchmarks use `asm volatile` to prevent dead-code elimination, while CPL preserves such loops by default. These tests should therefore be interpreted as loop-overhead measurements rather

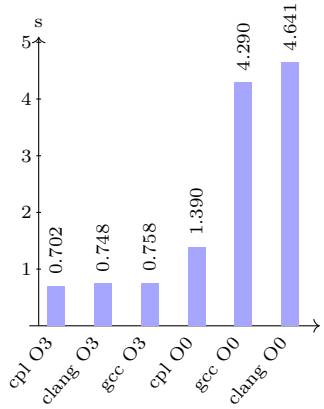

Figure 9: Empty loop benchmark. Optimized CPL assembly: 20 lines, down from 31.

Table 3: Documented CPL assembly-size reductions.

Benchmark	Before	After	Reduction
Empty loop	31	20	35.5%
Fibonacci	48	27	43.8%
String traversal	103	84	18.4%
Brainfuck	769	516	32.9%

than broad compiler-performance benchmarks. The documentation also recommends reading CPL timings conservatively, with an approximate +30% adjustment for possible measurement issues.

12 Benchmark Results

Lower runtime is better. Figures 9–12 reproduce the documented values. Table 3 reports assembly-size reductions after optimization.

The empty-loop result is best read as loop-overhead evidence because the benchmark is artificial and dead-code elimination behavior differs between implementations. Fibonacci reports optimized GCC and optimized CPL at 0.412 seconds, with optimized Clang at 0.417 seconds. String traversal reports optimized CPL slower than optimized Clang and GCC. Brainfuck reports optimized CPL and optimized Clang at

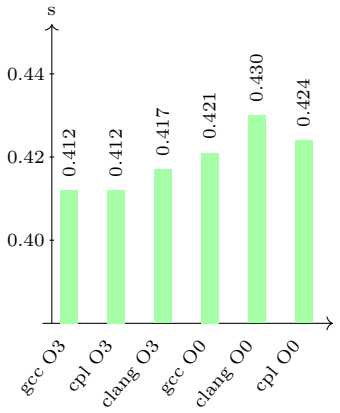

Figure 10: Million Fibonacci benchmark. Optimized CPL assembly: 27 lines, down from 48.

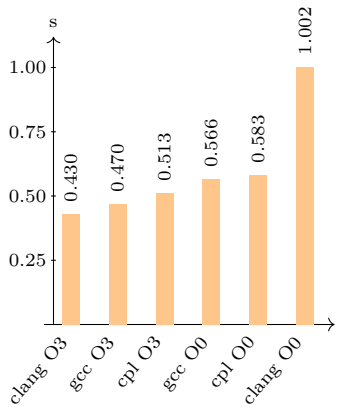

Figure 11: Pointer and string traversal benchmark. Optimized CPL assembly: 84 lines, down from 103.

0.403 seconds, with optimized GCC at 0.487 seconds.

13 Threats to Validity and Limitations

This report is a technical presentation of the project rather than an independent validation. The benchmark measurements have not been reproduced in a controlled external environment. The benchmark set is small, the programs are microbenchmarks, and timing methods differ between CPL and the C/C++ compilers. Some tests also use mechanisms such as `asm volatile`, which changes the optimization space and makes direct comparison less general.

The language itself is intentionally incomplete relative to industrial systems languages. It omits complex aggregate types and does not provide formal memory-safety guarantees. These omissions are part of the experimental design, but they also limit the current range of source programs that can be naturally expressed.

14 Conclusion

CPL and the Cordell Compiler form a compact experimental platform for systems-language and compiler-construction work. The language keeps a restricted C-like surface while exposing low-level control through explicit casts, pointers, annotations, system calls, and inline assembly. The compiler pipeline covers preprocessing, tokenization, semantic markup, AST construction, HIR lowering, CFG and SSA construction, DAG-based optimization, LIR, liveness analysis, register allocation, peephole rewriting, and NASM-oriented code generation.

The main value of the project is not that the language replaces production systems languages, but that it gives a coherent end-to-end environment for studying how source-level restrictions, compiler diagnostics, and low-level code generation interact. Future work should include a reproducible benchmark suite, clearer formalization of the language grammar and type rules, more precise alias analysis, and a broader evaluation of diagnostics on real low-level programs.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools.* 2nd edition, Addison-Wesley, 2007.

[2] A. W. Appel. *Modern Compiler Implementation in C.* Cambridge University Press, 1998.

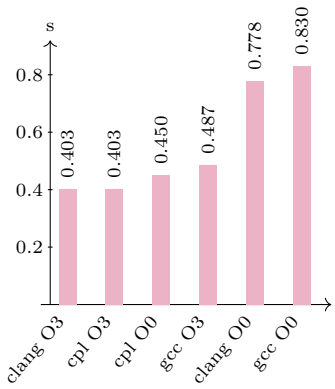

Figure 12: Brainfuck benchmark. Optimized CPL assembly: 516 lines, down from 769.

[3] K. D. Cooper and L. Torczon. *Engineering a Compiler*. 2nd edition, Morgan Kaufmann, 2012.

[4] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, 1991.

[5] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein. Register allocation via coloring. *Computer Languages*, 6(1):47–57, 1981.

[6] D. Kusswurm. *Modern X86 Assembly Language Programming*. 3rd edition, Apress, 2023.